

CHAPTER 5:

Methods, Classes & Objects

Property of German Malaysian Institute

Methods: Concept

Method also known as ‘function’ in C programming and ‘Procedure’ in VB.net

General definition of a method: a group of statements that perform a specific task.

Advantage using method → organize the coding

Methods in OOP

Methods : members of a class that provide a service for an object or perform some business logic.

✓ behavior of an object.

✓define the operations that can be performed in java programming.

✓ a method is invoked or called by a method call.

✓Method call specifies the name of the method and provides information (as arguments) that the called method requires to perform its task.

✓Method call completes, the method either returns a result to the calling method(caller) or returns control or caller.

✓ In general, method works like hierarchy form of job management.

Methods: Components

Modifiers—such as public, private

*The **return type***—the data type of the value returned by the method, or void if the method does not return a value.

*The **method name***—the rules for field names apply to method names as well, but the convention is a little different.

Parameter list in parenthesis—a comma-delimited list of input parameters, preceded by their data types, enclosed by parentheses, (). Empty parentheses: there are no parameters.

Method body, enclosed between braces—the method's code, including the declaration of local variables, goes here.

Methods: Components

```
public static int method1(int num1, String strname, int
num2)

public static void method2(int num2, String strname2)

public static void method3( )
```

Modifier	Apply in Java, not apply in JApplet	Return type	Method Name	Parameters
public	static	int	Method1	(int num1, String strname, int num2)
public	static	void	Method2	(int num2, String strname2)
public	static	void	Method3	( ) → none

Property of German Malaysian Institute

Types of Methods

- Example, initializing 4 new methods
  - Type 1 → have a return type, and have pass argument
  - Type 2 → have no return type/void
    → have pass argument
  - Type 3 → No return type/void, no pass argument
  - Type 4 → Have return type, no pass argument

Property of German Malaysian Institute

EXAMPLES OF DIFFERENT TYPES OF METHOD DEFINITION

```
public class Main
{
        public Main()
        {
        }

        public static void main(String[] args)
        {
        }

        public static int method1(int num1, String strname, int num2)
        {
        }

        public static void method2(int num2, String strname2)
        {
        }

        public static void method3( )
        {
        }

        public static int method4( )
        {
        }

}
```

Method body

Property of German Malaysian Institute

Methods

✓ Example: Math-Class Methods

Math.sqrt(900)

✓ Method declaration:

Return-value-type method name(param1,param2,…,paramN)
{
declarations and statements
}

Example:

```
Public double maximum(double x, double y, double z)
{
   return Math.max( x, Math.max( y , z ) );
}
```

Method Signature

✓ method signature: the method's name and the parameter types.

✓ Example method signature:

maximum(double , double , double)

```
Public void init()  {
.
.
      double max = maximum(number1, number2, number3);
.
.}
public double maximum(double x, double y, double z)
{
   return Math.max( x, Math.max( y , z ) );
}
```

Methods – Pass Arguments & Return Value

```
public static void main () (String[] args)
{
int value;
int number1 = 35;
int number2 = 17;
String name = "Alfonso";
value = method1(number1,name,number2);
System.out.println ("The total value is " + value);
 }
```

Calling method1

```
   public static int method1(int num1, String strname, int num2)
   {
      int total;
      total = num1 + num2;
      System.out.println ("Your name is " + strname);
      return total;
   }
```

Returning the calculated value

Property of German Malaysian Institute

Methods – Exercises

- Initialize a method name **Testing**, with **public** modifier, with no parameters and no return type
- Initialize a method name **Calculate**, with **public** modifier, with no parameters and return type **int**
- Initialize a method name **Display** with **public** modifier, with parameters **int1** (float data type) and return type **String**

Property of German Malaysian Institute

Methods Overloading

- Java allows several methods of the same name declared in the same class as long as the methods have different set of parameters.

```
public static void draw(String s) {
        ... }

public static void draw(int i) {
        ... }

public static void draw(double f) {
        ... }

public static void draw(int i, double f) {
        ... }
```

Property of German Malaysian Institute

Introduction to Classes & Objects

Realworld → Various objects with a similar kind



Property of German Malaysian Institute

Class: Concept

- A set of objects that share a common structure and behavior.
- Blueprint for creating different objects which defines its properties and behaviors
- Classes have 2 members: **fields**(Field variables) and **methods**

```
public class MainClass
{
        private int  aField;

        public  MainClass( )
        {
        }

        public static void aMethod(
)
        {
        }
}
```

Field variable

Constructor

Methods

Class: Example

```
public class SmartPhone
{
	String manufact, model;
	double storage,  screenSize;

public static void insertRecord(String man, String mod)
	{
		manufacturer = man;
		model = mod
	}
}
```

Property of German Malaysian Institute

Class (cont..)

- Initializing several classes in a program

```
public class MainClass{
        private int  aField;

        public static void aMethod( ) {
        }
        public static void main(String[] args){
                System.out.println("Application C entry point");
        }
}

public class Secondclass
{
        private int  number;

        public static void calculated( )
        {
        }
}
```

Property of German Malaysian Institute

Multiple Classes

- Initializing several classes in a program

```
class A
{
}
class B
{
}
class C {
public static void main(String[] args)
{
 System.out.println("Application C entry point");
 }
}
```

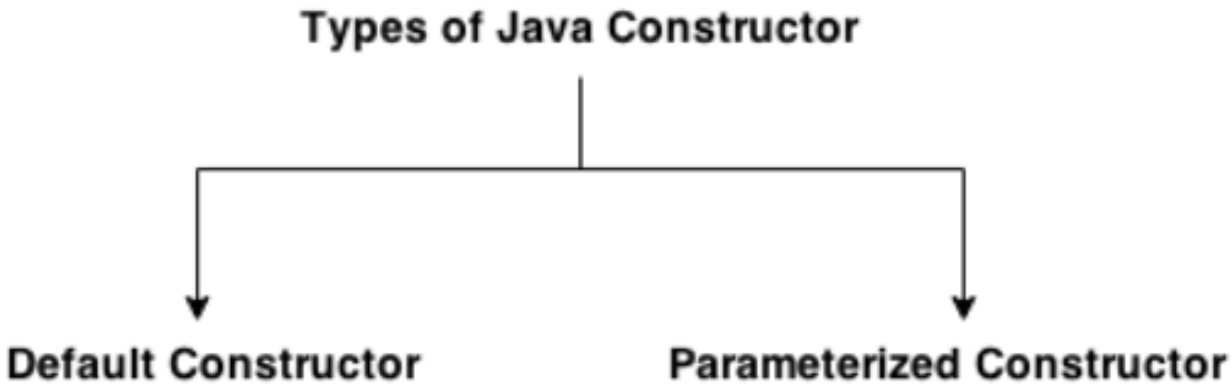
Property of German Malaysian Institute

Class - Example

```
public class Bicycle {
 // the Bicycle class has three fields
    public int cadence;
    public int gear;
   public int speed;
// the Bicycle class has one constructor
   public Bicycle(int startCadence, int startSpeed, int startGear) {
                gear = startGear;
                cadence = startCadence;
                speed = startSpeed; }
 // the Bicycle class has four methods
   public void setCadence(int newValue) {
                cadence = newValue; }
   public void setGear(int newValue) {
                gear = newValue; }
    public void applyBrake(int decrement) {
                speed -= decrement; }
    public void speedUp(int increment) {
```

Class - Constructor

- A code block to initialize an object.
- Constructor declarations look like method declarations—except that they use the name of the class and have no return type.
- Constructor is invoked when an object is created.



Class – Constructor (cont)

Constructor VS Method

Java Constructor	Java Method
Constructor is used to initialize the state of an object.	Method is used to expose behaviour of an object.
Constructor must not have return type.	Method must have return type.
Constructor is invoked implicitly.	Method is invoked explicitly.
The java compiler provides a default constructor if you don't have any constructor.	Method is not provided by compiler in any case.
Constructor name must be same as the class name.	Method name may or may not be same as class name.

Class – Constructor (cont)

Parameterized Constructor – constructor with parameters

```
public Bicycle(int startCadence, int startSpeed, int startGear)
{     gear = startGear;
      cadence = startCadence;
      speed = startSpeed; }
```

To create a new Bicycle object called myBike, a constructor is called by the new operator: **Bicycle myBike = new Bicycle(30, 0, 8);**

Default constructor - constructor with no parameters:

```
public Bicycle() {
gear = 1; cadence = 10; speed = 0; }
```

Bicycle yourBike = new Bicycle();

Invokes the no-argument constructor to create a new Bicycle object called yourBike.

Class – Constructor(Example)

```
public class SmartPhone
{
      String manufact, model;
      double storage,  screenSize;

public SmartPhone(String man, String mod, double st,
double sc)
{
      manufact = man;
      model = mod;
      storage = st;
      screenSize = sc;
}
}
```

Class – Constructor(Example 2)

A class could have more than 1 constructor

```
public class SmartPhone
{
      String manufact, model;
      double storage,  screenSize;

public SmartPhone(String man, String mod, double st,
double sc)
{
      manufact = man;
      model = mod;
      storage = st;
      screenSize = sc;
}
Public SmartPhone(){
}
}
```

Class –Exercise 1

- Define a class name **main**, complete with **constructor**, and a method name **Start**
- Define another method name **Second** with two parameters and no return type, use any modifier

- Define another class name **Calculator**, complete with **constructor**
- Define a method name **Third** with two parameters and integer as return type, use any modifier
- Define another method name **Fourth** with no parameter and no return type, use any modifier

Property of German Malaysian Institute

Class –Exercise 2

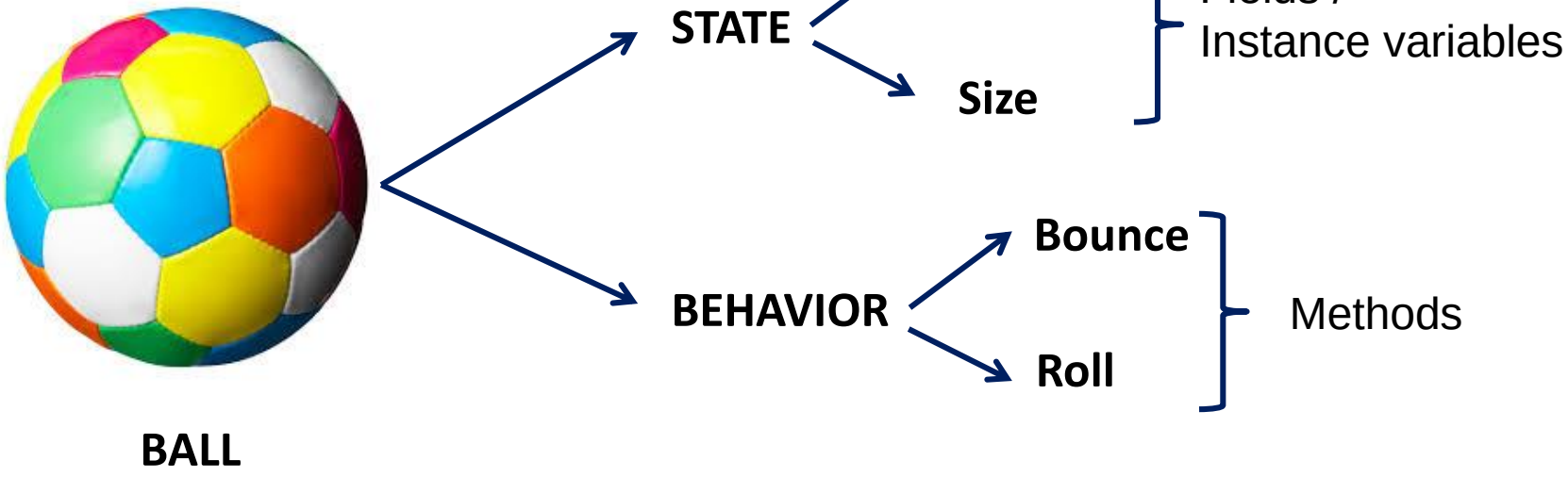
- Initializing a class name **Robot**, complete with **constructor**, and a method name **main**
- Initialize another method name **Walking** with **int and float parameter** and **return type char**, use any modifier

- Initialize another class name **Body** , complete with **constructor**
- Initialize a method name **setBody** with no parameter and return **type int**, use any modifier
- Initialize another method name **setMaterial** with **parameter char** and no return type, use any modifier

Property of German Malaysian Institute

Object: Concept

- ✓Instance of a particular class.
- ✓Has state and behavior.
- ✓ A class could contains many objects.
- ✓Each object has its own set of data.



Object: Declare, instantiate & initialize

Creating Object

Class_Name variable; // declaration of class variable

Instantiation

Class_Variable = new Class_Name(); // create an object for a class

OR

Class_name variable = new Class_Name ()

Initialization

```
Bicycle myBike = new Bicycle(30, 0, 8);
Bicycle yourBike = new Bicycle();
```

Object (cont)

- ✓ The variables that the object contains are called instance variables.

Referencing an Object's Fields

objectReference.fieldName → `myBike.gear`

- ✓ The subroutines that the object contains are called instance methods.
- ✓ Objects interact by invoking methods that allows a program to carry out various tasks.

Calling an Object's Methods

objectReference.methodName(argumentList);

or

objectReference.methodName(); → `myBike.setgear(4)`

Object - example

```
public class MainClass
{
        public static void main( String args[] )
        {
        GradeBook myGradeBook = new GradeBook();
        myGradeBook.displayMessage();
        }
}

class GradeBook
{
        public void displayMessage()
        {
        System.out.println( "Welcome to the Grade Book!" );
        }

}
```

Initialize an object

Data Type is GradeBook

Object name is myGradeBook

Use the Object Method

Method name is displayMessage